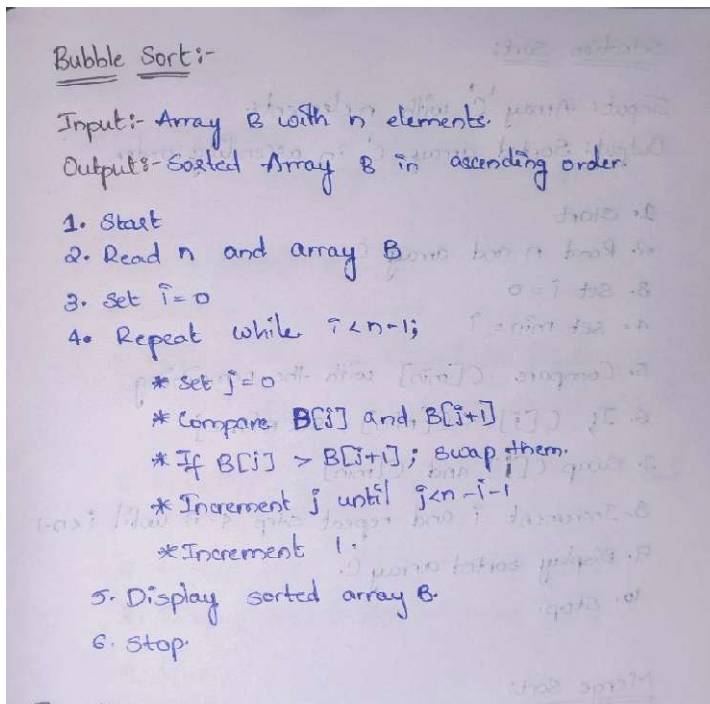


PRACTICAL: 01

Aim: To implement and find the time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.

A) Bubble Sort:

Algorithm:



Program / Code:

```
#include<iostream>

using namespace std;

int main()
{
    int a[60], size, i, j, temp;

    cout<<"Enter the size of the array: ";

    cin>>size;

    cout<<"Enter the array elements: ";

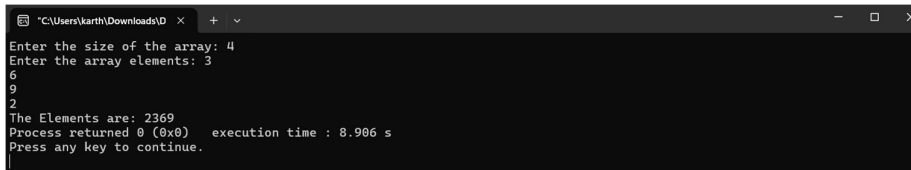
    for(i=0; i<size; i++){
```

```
        cin>>a[i];
    }
    for(i=0; i<size-1; i++){
        for(j=0; j<size-i-1; j++){
            if(a[j]>a[j+1]){
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }

    cout<<"The Elements are: ";
    for(i=0; i<size; i++){
        cout<<a[i];
    }

    return 0;
}
```

Output:



```
Enter the size of the array: 4
Enter the array elements: 3
6
9
0
2
The Elements are: 2369
Process returned 0 (0x0)   execution time : 8.906 s
Press any key to continue.
```

Time Complexity:

Bubble Sort:

```

for (i=0; i < n-1; i++) {
    for (j=0; j < n-i-1; j++) {
        if (a[j] > a[j+1]) {
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }
    }
}

```

Case (worst case) (mode)

$$T(n) = (n-1) + (n-2) + \dots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

→ Best case = Average case = worst case = $O(n^2)$

B) Insertion Sort:

Algorithm:

Insertion Sort:

Input:- Array A with n elements
Output:- Sorted array A in ascending order

1. Start
2. Read n and array A.
3. Set $i = 1$
4. Set $Key = A[i]$ and $j = i - 1$
5. While $j \geq 0$ and $A[j] > Key$, shift $A[j]$ to $A[j+1]$
6. Insert Key at $A[j+1]$
7. Increment i and repeat steps 4-6 until $i < n$
8. Display sorted array A.
9. Stop.



Program / Code:

```
#include<iostream>

using namespace std;

int main(){

    int a[60], size, i, j, min, temp;

    cout<<"Enter the size of the array: ";

    cin>>size;

    cout<<"Enter the array elements: ";

    for(i=0; i<size; i++){

        cin>>a[i];

    }

    for(i=0; i<size-1; i++){

        min = i;

        for(j=i+1; j<size; j++){

            if(a[j] < a[min]){

                min = j;

            }

        }

        if(min != i){

            temp = a[i];

            a[i] = a[min];

            a[min] = temp;

        }

    }

    cout<<"The Elements are: ";

    for(i=0; i<size; i++){

        cout<<a[i]<<" ";

    }

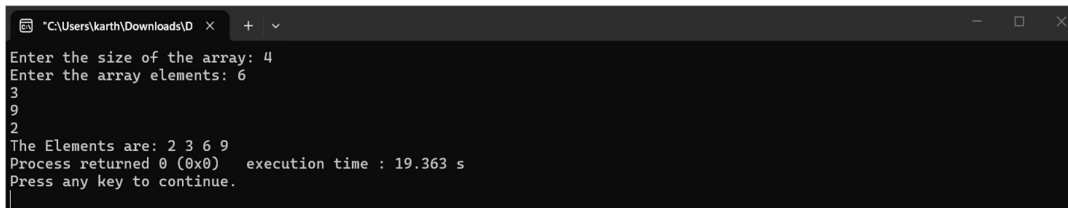
}
```

```

    }
    return 0;
}

```

Output:



```

C:\Users\karth\Downloads\ID >
Enter the size of the array: 4
Enter the array elements: 6
3
9
2
The Elements are: 2 3 6 9
Process returned 0 (0x0)   execution time : 19.363 s
Press any key to continue.

```

Time Complexity:

Insertion Sort

```

for (i = 1; i < n; i++) {
    key = arr[i]
    j = i - 1;
    while (j >= 0 && arr[j] > key) {
        arr[j+1] = arr[j]
        j = j - 1;
    }
    arr[j+1] = key;
}

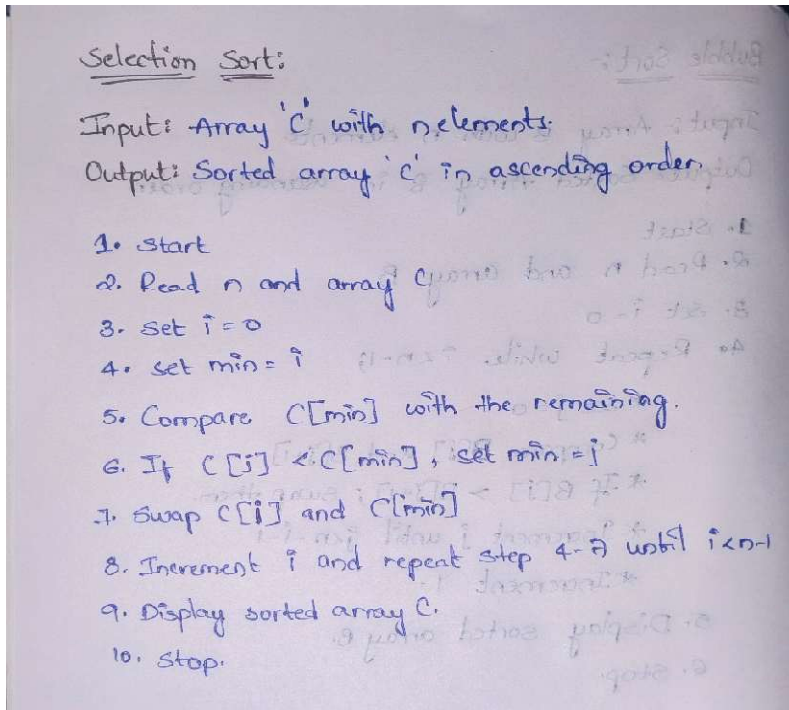
```

$\leftarrow n$
 $\leftarrow 1$
 $\leftarrow 1$
 $\leftarrow 1$
 $\leftarrow 1$
 $\leftarrow 1$
 $\leftarrow 1$

$T(n) = n + 1 + 1 + 1 + 1 + 1 + 1$
 $T(n) = n + 6$
 $T(n) = O(n)$

C) Selection Sort:

Algorithm:



Program / Code:

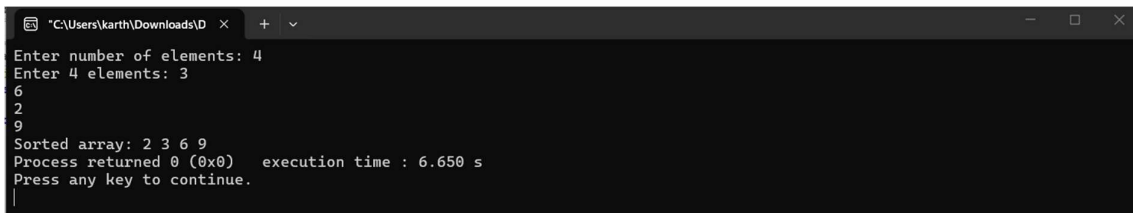
```
#include <iostream>

using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
```

```
        arr[minIndex] = arr[i];  
        arr[i] = temp;  
    }  
}  
  
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    selectionSort(arr, n);  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++){  
        cout << arr[i] << " ";  
    }  
    return 0;  
}
```

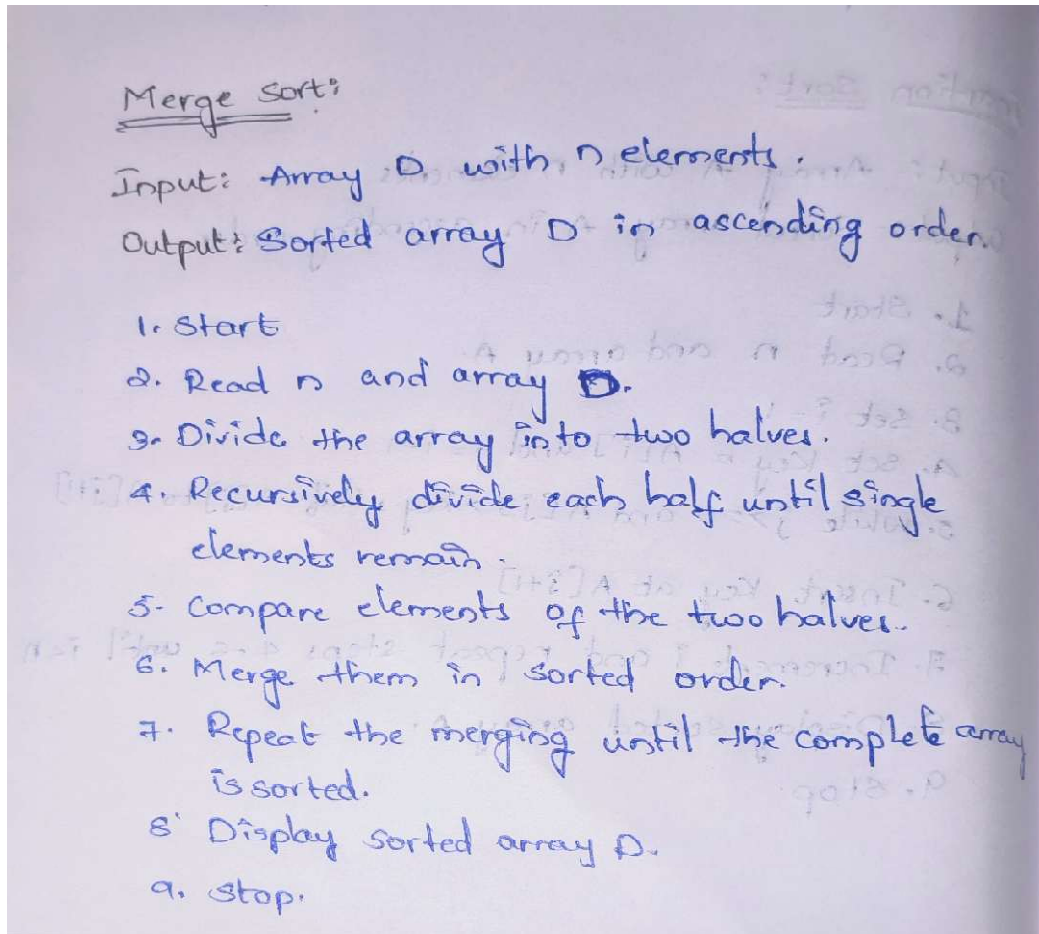
Output:



```
*C:\Users\karth\Downloads\ID x + v  
Enter number of elements: 4  
Enter 4 elements: 3  
6  
2  
9  
Sorted array: 2 3 6 9  
Process returned 0 (0x0) execution time : 6.650 s  
Press any key to continue.
```


D) Merge Sort:

Algorithm:



Program / Code:

```
#include <iostream>

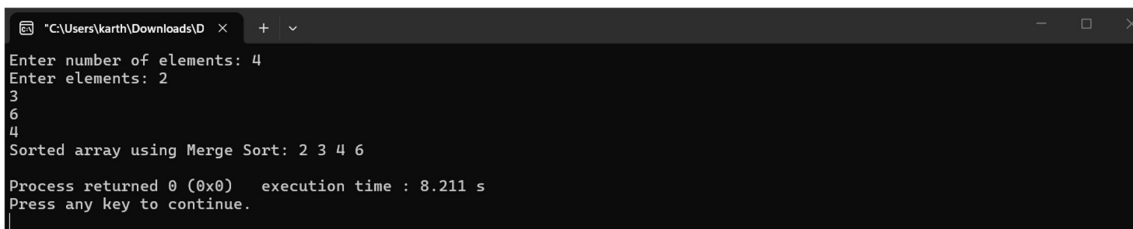
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
```

```
    }  
    for (int j = 0; j < n2; j++) {  
        R[j] = arr[mid + 1 + j];  
    }  
    int i = 0;  
    int j = 0;  
    int k = left;  
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) {  
            arr[k++] = L[i++];  
        } else {  
            arr[k++] = R[j++];  
        }  
    }  
    while (i < n1) {  
        arr[k++] = L[i++];  
    }  
    while (j < n2) {  
        arr[k++] = R[j++];  
    }  
}  
  
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}
```

```
    }  
}  
  
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    mergeSort(arr, 0, n - 1);  
    cout << "Sorted array using Merge Sort: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:



```
"C:\Users\karth\Downloads\ID" x + v  
Enter number of elements: 4  
Enter elements: 2  
3  
6  
4  
Sorted array using Merge Sort: 2 3 4 6  
Process returned 0 (0x0) execution time : 8.211 s  
Press any key to continue.
```



Time Complexity:

Merge Sort

no. of divisions: $\log_2 n$ ($2T(n/2)$)

merger: each level merges all n elements
 $O(n)$

Total time: $T(n) = 2T(n/2) + n$.

$T(n) = 2T(n/2) + n \rightarrow \textcircled{1}$

$n \rightarrow n/2$ put in $\textcircled{1}$

$T(n/2) = 2T(n/4) + \frac{n}{2}$

$T(n) = 2T(n/4) + \frac{n}{2} \rightarrow \textcircled{2}$

$\textcircled{2}$ in $\textcircled{1}$

$T(n) = 2(2T(n/4) + \frac{n}{2}) + n$

$= 2^2 T(n/4) + \frac{2n}{2} + n$

$T(n) = 2^2 T(n/4) + 2n \rightarrow \textcircled{3}$

$n \rightarrow n/4$ put in $\textcircled{3}$

$T(n/4) = 2^2 T(n/8) + 2(\frac{n}{4}) \rightarrow \textcircled{4}$

$\textcircled{4}$ in $\textcircled{3}$

$T(n) = 2^2 \left(2^2 T(n/8) + 2(\frac{n}{4}) \right) + 2n$

$= 2^4 T(n/8) + 4n$

$T(n) = 2^k T(n/2^k) + kn$

$T(n) = 2^k T(n/2^k) + kn$

$\frac{n}{2^k} = 1 \quad n = 2^k$

Apply log both sides

$\log n = \log 2^k$

$k = \log n$

$T(n) = 2^{\log n} T\left(\frac{n}{2^{\log n}}\right) + \log n(n)$

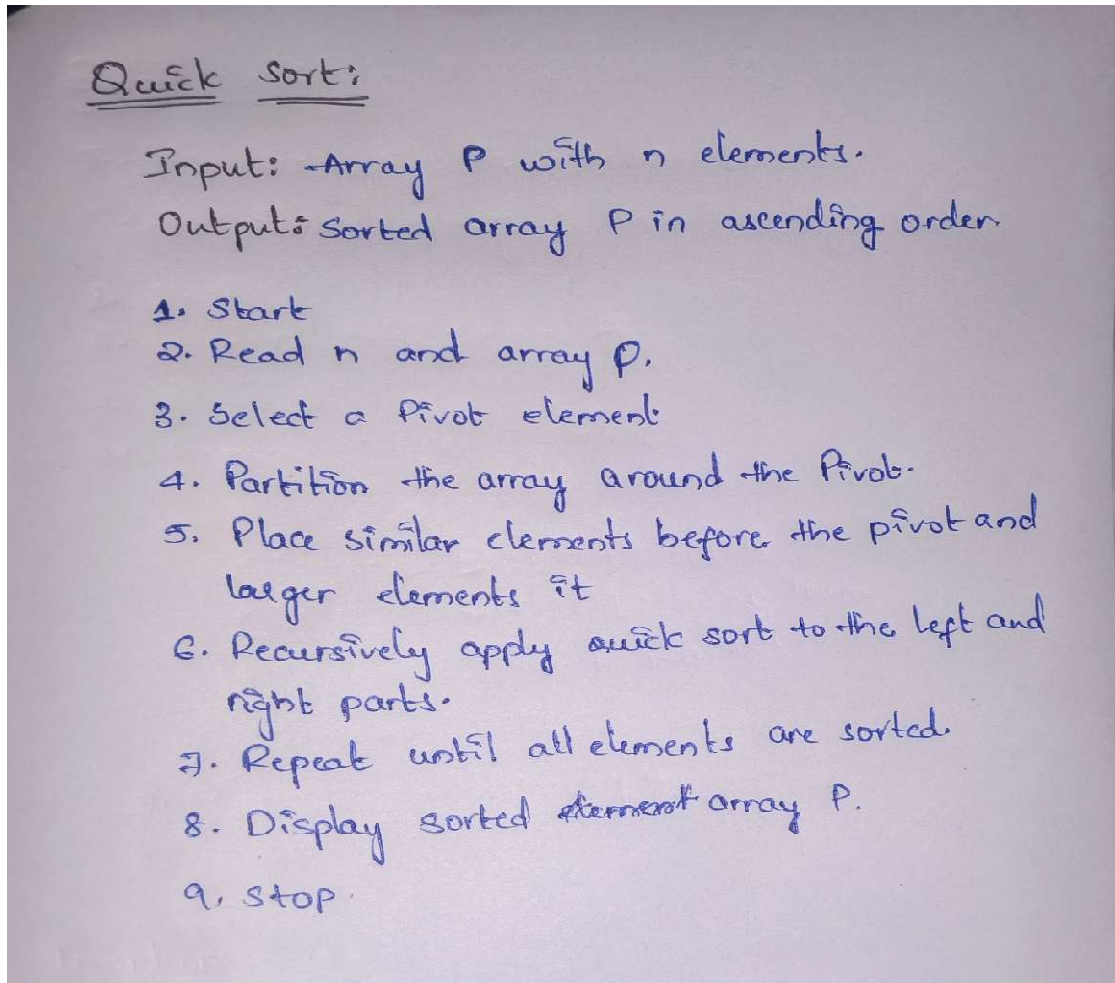
$= n \log^2 + \frac{n}{n^{\log 2}} + \log n(n)$

$= n(\log n + 1)$

$T(n) = O(n \log n)$

E) Quick Sort:

Algorithm:



Program / Code:

```
#include <iostream>
using namespace std;
int partition(int arr[], int lb, int ub) {
    int pivot = arr[lb];
    int start = lb;
```



```
int end = ub;
while (start < end) {
    while (arr[start] <= pivot) {
        start++;
    }
    while (arr[end] > pivot) {
        end--;
    }
    if (start < end) {
        swap(arr[start], arr[end]);
    }
}
swap(arr[lb], arr[end]);
return end;
}

void quickSort(int arr[], int lb, int ub) {
    if (lb < ub) {
        int loc = partition(arr, lb, ub);
        quickSort(arr, lb, loc - 1);
        quickSort(arr, loc + 1, ub);
    }
}

int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;
    int arr[n];
```




```
cout << "Enter " << n << " elements: ";  
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}  
quickSort(arr, 0, n - 1);  
cout << "Sorted array using Quick Sort: ";  
for (int i = 0; i < n; i++) {  
    cout << arr[i] << " ";  
}  
return 0;  
}
```

Output:

```
*C:\Users\karth\Downloads\ID x + v  
Enter size of array: 4  
Enter 4 elements: 9  
3  
5  
6  
Sorted array using Quick Sort: 3 5 6 9  
Process returned 0 (0x0) execution time : 9.644 s  
Press any key to continue.  
|
```

Time Complexity:

Quick sort.

The pivot divides the array into two equal halves
 $\rightarrow 2T(n/2)$
 for sorting we use n elements at last $O(n)$

$$T(n) = 2T(n/2) + n \rightarrow (1)$$

$n \rightarrow n/2$ put in (1)

$$T(n/2) = 2T(n/4) + n/2$$

$$T(n) = 2T(n/4) + n/2 \rightarrow (2)$$

sub (2) in (1)

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$= 2^2 T(n/4) + 2n + n$$

$$T(n) = 2^2 T(n/4) + 3n \rightarrow (3)$$

$n \rightarrow n/4$ put in (3)

$$T(n/4) = 2^2 T(n/8) + 2(n/4) \rightarrow (4)$$

(4) in (3)

$$T(n) = 2^3 \left[2^2 T(n/8) + 2(n/4) \right] + 3n$$

$$= 2^4 T(n/8) + 4n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$n/2^k = 1 \Rightarrow n = 2^k \Rightarrow$ Apply log on both sides

$$\log n = \log 2^k \Rightarrow k = \log n$$

$$T(n) = 2^{\log n} T(n/2^{\log n}) + \log n(n)$$

$$= n^{\log 2} + \frac{n}{n^{\log 2}} + \log n(n)$$

$$= n \log n + n$$

$$T(n) = O(n \log n)$$

Conclusion:

In this practical, we studied and implemented five sorting algorithms: Bubble, Selection, and Insertion Sort are simple but slow for large data ($O(n^2)$ time), making them suitable only for small lists. Merge Sort and Quick Sort are much faster for large data since they run in $O(n \log n)$ time. So, simple sorts work fine for small lists, but Merge/Quick Sort are better choices for big data.